

# 检测和评估API文档

## 概述

新增的检测和评估API提供了智能的图像分析流程：先进行目标检测，如果检测到种子对象则进行成分分析，否则返回未检测到种子对象的信息。这种设计可以避免对不包含种子的图像进行无意义的成分分析。

## API端点

### 1. 检测和评估接口

**端点:** POST /v1/detect-and-eval

**功能:** 先进行目标检测，如果检测到种子对象则进行成分分析

### 请求参数

```
{
  "img_src": "string",           // 必需: 图像文件的URL或路径
  "model_name": "string",        // 可选: 用于成分分析的模型名称, 默认为'FasterNet'
  "conf_threshold": "float|null", // 可选: 检测置信度阈值, null表示使用配置默认值(0.25)
  "iou_threshold": "float|null"  // 可选: IoU阈值, null表示使用配置默认值(0.45)
}
```

### 支持的模型名称:

- MPViT
- ResNet
- FasterNet (默认)
- EfficientNet
- Swin
- VanillaNet

## 响应格式

```
{
  "success": "boolean",           // 整体操作是否成功
  "stage": "string",              // 执行阶段
  "message": "string",            // 状态消息
  "detected": "boolean",          // 是否检测到对象
  "detection_result": "object",   // 检测结果详情
  "evaluation_result": "object",  // 成分分析结果（如果有）
  "total_time_delta": "float",    // 总耗时（秒）
  "image_hash": "string",         // 图像哈希值
  "model_name": "string"          // 使用的模型名称
}
```

## 执行阶段说明

- "detection" : 检测阶段失败
- "detection\_only" : 仅完成检测，未检测到对象
- "complete" : 检测和分析都完成
- "evaluation" : 检测成功但分析失败
- "image\_loading" : 图像加载失败
- "processing" : 处理过程失败

## 使用示例

### 1. 基本使用（默认参数）

```
curl -X POST "http://localhost:8000/v1/detect-and-eval" \
  -H "Content-Type: application/json" \
  -d '{
    "img_src": "tests/test_images/image_custom.png"
  }'
```

响应示例（未检测到对象）：

```
{
  "success": true,
  "stage": "detection_only",
  "message": "未检测到种子对象",
  "detected": false,
  "detection_result": {
    "success": true,
    "detected": false,
    "objects": [],
    "detection_count": 0,
    "time_delta": 0.2264,
    "conf_threshold": 0.25,
    "iou_threshold": 0.45
  },
  "evaluation_result": null,
  "total_time_delta": 0.2264,
  "image_hash": "abc123...",
  "model_name": "FasterNet"
}
```

## 2. 自定义参数使用

```
curl -X POST "http://localhost:8000/v1/detect-and-eval" \
-H "Content-Type: application/json" \
-d '{
  "img_src": "path/to/seed_image.jpg",
  "model_name": "ResNet",
  "conf_threshold": 0.3,
  "iou_threshold": 0.5
}'
```

响应示例（检测到对象并完成分析）：

```
{
  "success": true,
  "stage": "complete",
  "message": "检测和分析完成",
  "detected": true,
  "detection_result": {
    "success": true,
    "detected": true,
    "objects": [
      {
        "confidence": 0.85,
        "class_id": 0,
        "class_name": "seed",
        "bbox": [100, 150, 300, 350]
      }
    ],
    "detection_count": 1,
    "time_delta": 0.15,
    "conf_threshold": 0.3,
    "iou_threshold": 0.5
  },
  "evaluation_result": {
    "protein": 45.23,
    "oil": 52.67,
    "time_delta": 1.234,
    "memory_cost": 256.5,
    "seed": 123,
    "seed_info": {...}
  },
  "total_time_delta": 1.384,
  "image_hash": "def456...",
  "model_name": "ResNet"
}
```

### 3. Python客户端示例

```
import requests

# 基本使用
def detect_and_eval_image(image_path, model_name="FasterNet"):
    url = "http://localhost:8000/v1/detect-and-eval"
    data = {
        "img_src": image_path,
        "model_name": model_name
    }

    response = requests.post(url, json=data)

    if response.status_code == 200:
        result = response.json()

        if result["success"]:
            if result["detected"]:
                print(f"检测到 {result['detection_result']['detection_count']} 个对象")
                if result["evaluation_result"]:
                    eval_result = result["evaluation_result"]
                    print(f"蛋白质: {eval_result['protein']:.2f}%")
                    print(f"油脂: {eval_result['oil']:.2f}%")
            else:
                print("未检测到种子对象")
        else:
            print(f"分析失败: {result.get('error', '未知错误')}")
    else:
        print(f"请求失败: {response.status_code}")

    return response.json()

# 使用示例
result = detect_and_eval_image("my_seed_image.jpg", "ResNet")
```

# 内部方法

## ModelAPI新增方法

### 1. detect\_objects()

```
def detect_objects(self, image, conf_threshold: float = None, iou_threshold: float = None) -> dict:
    """
    对图像进行目标检测

    Args:
        image: 输入图像
        conf_threshold: 置信度阈值，默认使用配置中的值
        iou_threshold: IoU阈值，默认使用配置中的值

    Returns:
        检测结果字典，包含是否检测到对象和检测详情
    """
```

### 2. detect\_and\_eval()

```
def detect_and_eval(self, image, model_name: str = 'FasterNet',
                    conf_threshold: float = None, iou_threshold: float = None) -> dict:
    """
    先进行目标检测，如果检测到对象则进行成分分析

    Args:
        image: 输入图像
        model_name: 用于成分分析的模型名称
        conf_threshold: 检测置信度阈值
        iou_threshold: 检测IoU阈值

    Returns:
        包含检测和分析结果的字典
    """
```

## 配置参数

相关配置在 config.py 中的 MODEL\_CONFIG :

```
MODEL_CONFIG = {  
    # ... 其他配置 ...  
    # 检测模型配置  
    "detect_model": "YOLO",  
    "detect_model_path": "weights/yolov8n.pt",  
    "detect_conf_threshold": 0.25,  
    "detect_iou_threshold": 0.45  
}
```

## 错误处理

### 常见错误情况

#### 1. 图像加载失败

```
{  
    "success": false,  
    "stage": "image_loading",  
    "error": "图像获取失败：文件不存在"  
}
```

#### 2. 检测模型加载失败

```
{  
    "success": false,  
    "stage": "detection",  
    "error": "检测模型加载失败"  
}
```

#### 3. 成分分析失败

```
{  
    "success": false,  
    "stage": "evaluation",  
    "error": "模型预测失败：CUDA内存不足",  
    "detected": true,  
    "detection_result": {...}  
}
```

# 性能特点

- 1. **智能流程**: 只有检测到种子对象才进行成分分析，避免无效计算
- 2. **并行优化**: 检测和分析使用不同的模型，可以独立优化
- 3. **内存管理**: 自动卸载模型，避免内存泄漏
- 4. **时间统计**: 提供详细的时间消耗统计
- 5. **错误恢复**: 完善的错误处理和状态反馈

# 与原有API的对比

| 特性   | /predict  | /v1/detect-and-eval |
|------|-----------|---------------------|
| 功能   | 直接成分分析    | 检测+条件分析             |
| 适用场景 | 确定包含种子的图像 | 未知内容的图像             |
| 计算效率 | 固定计算量     | 智能计算量               |
| 错误处理 | 基础错误处理    | 分阶段错误处理             |
| 返回信息 | 分析结果      | 检测+分析结果             |

# 测试

运行测试脚本验证功能：

```
# 测试内部方法
python test_detect_and_eval_api.py

# 测试API端点（需要先启动服务器）
python test_fastapi_endpoints.py
```

# 总结

新的检测和评估API提供了更智能的图像分析流程，能够：

- 1. **提高效率**: 避免对不包含种子的图像进行无意义的成分分析
- 2. **增强鲁棒性**: 分阶段处理，提供详细的错误信息
- 3. **保持兼容**: 不影响原有的predict API



#### 4. **扩展性强:** 支持自定义检测参数和分析模型

这为系统提供了更加智能和高效的图像分析能力！